

OpenRouter CSM Technical Case Study

Preparation pack: strategy, account profile, technical coaching, and the benchmark tooling

Prepared for Ben Lemmers | September 2026 | 20 minute retention presentation plus 5 to 10 minutes Q and A

The brief: you are the CSM. Ninety days from renewal on a top account, a lead developer emails you claiming OpenRouter adds unnecessary latency versus going direct to Anthropic, and the CTO is copied. You have 20 minutes to address the developer technically and make the business case to the CTO, then 5 to 10 minutes of hostile questions.

Contents

1. What is actually being scored
2. The strategic spine
3. The 20 minute time budget
4. The account: Kestrel AI
5. Technical coaching: latency mechanics
6. Published figures and how to use them
7. The demo plan
8. Benchmark script: setup and run instructions
9. Q and A preparation
10. Traps to avoid
11. Next steps

1. What is actually being scored

The prompt looks like a product knowledge test. It is not. Four things are being measured.

1. **Two audiences, one room.** The developer needs technical credibility and a fix. The CTO needs risk and money. Most candidates pick one and lose the other.
2. **Whether you concede the true part.** A router adds a hop. If you claim otherwise you are done in the first two minutes.
3. **Whether you read the escalation.** A lead developer emailing the CSM with the CTO copied, 90 days from renewal, is not a support ticket. Someone is building an internal case to go direct. Latency is the stated reason, not necessarily the real one.
4. **Time.** They told you time management is an evaluation factor. Build 15 minutes of content for a 20 minute slot, because your stakeholders will interrupt.

2. The strategic spine

Do not defend the milliseconds. Reframe the axis.

Direct to Anthropic wins on the happy path median. That is true and you should say it out loud on an early slide. The comparison that matters to a production system is p95 and p99 *including* retries, rate limit responses, capacity events and provider incidents. One rate limit wave or a single provider degradation erases months of saved median milliseconds, and going direct means the fallback logic becomes the customer's engineering backlog.

Then give the developer real fixes, not reassurance: pin the provider order for the hot path, exclude slow endpoints, turn on streaming and prompt caching, use BYOK so they keep their Anthropic commit, discounts and rate limits while keeping the gateway, and measure with generation records rather than wall clock guesses.

THE MOVE THAT WINS THIS SCENARIO

Concede the split. Let the one latency critical surface go direct or pinned, keep everything else on OpenRouter. You kill the all or nothing frame, you look honest, and you keep roughly 90 percent of the spend. The CTO case is then about that 90 percent: switching cost, optionality as model prices fall, one bill and one integration surface, spend attribution, and who owns failover at 2am.

3. The 20 minute time budget

Time	Segment
0:00 to 1:30	What I did after your email, and what I am assuming
1:30 to 6:30	Where the milliseconds actually go, measured
6:30 to 11:00	The fix plan, aimed at the developer, with a 30 day pilot
11:00 to 16:00	The business case, aimed at the CTO
16:00 to 18:00	30/60/90 with owners and dates, plus the renewal ask
18:00 to 20:00	Buffer, because you will not get a clean run

Open with evidence of pre work, not with a title slide. Something like: I replied to Raj within the hour, pulled your last 30 days of traffic, and ran a benchmark against your top three routes. That buys you the room.

4. The account: Kestrel AI

The case is hypothetical, so you have to invent the account. Rename it if you like, but use these numbers consistently across every slide, and put them on an assumptions slide up front so the panel can correct you.

Who they are	Series C agentic customer support platform, about 600 employees. Their product suggests replies to human agents in real time and auto deflects tier 1 tickets for enterprise contact centers.
Contract	OpenRouter Enterprise since September 2024. Renews December 15, 2026. Roughly 4.1 million dollars annualized inference run rate, about 340 thousand a month at list.
Volume	42 million requests a month across two very different workloads.
Hot path	Agent Assist. 8 million requests a month, streaming, p95 time to first token target under 900 ms. This is the piece their own enterprise customers see, and it sits under an SLA.
Cold path	34 million requests a month. Overnight QA scoring, ticket summarization, eval runs. Latency is irrelevant here.
Model mix	68 percent Claude for agent assist, 18 percent a cheap open weight model for classification and PII redaction, 9 percent GPT class for summarization, 5 percent embeddings and miscellaneous.
People	Raj Mehta, Lead Platform Engineer, wrote the email. Dana Whitfield, CTO, is copied. The VP Engineering who signed the original deal has since left, which is why nobody internally is defending the decision.
Trigger	Raj ran 200 calls from his laptop three weeks ago and measured a 140 ms median gap versus Anthropic direct. Two weeks before that, a provider capacity event caused a rough afternoon. He wants to move the hot path direct and write his own fallback logic.

THE NUMBER THAT CARRIES THE WHOLE PITCH

The hot path is 8 million of 42 million requests, roughly 19 percent of volume, but it carries 100 percent of the executive attention. Your case lives or dies on whether you see that distinction and say it out loud.

5. Technical coaching: latency mechanics

5.1 The four numbers that matter

Most people say latency and mean four different things. Separating them is your credibility moment.

Metric	What it is and what drives it
Time to first token	Request sent to first token back. This is what a streaming interface feels like. Driven by network hops, queueing at the provider, and prefill, which scales with prompt size.
Inter token latency	Also called decode speed, in tokens per second. Set entirely by the provider's serving hardware and batching. A gateway does not change this unless it buffers the stream instead of relaying it.
Total completion time	Time to first token plus output tokens times decode time. A 600 token answer at 50 tokens per second is about 12 seconds. A 140 ms routing difference is roughly 1 percent of that.
Percentiles	p50 is what Raj felt on his laptop. p95 and p99 are what Kestrel's users feel, and the tail is driven by rate limits, retries and provider degradation, not by a network hop.

Your line to the developer: routing overhead lands almost entirely in the first bucket, and that bucket is usually a small share of total completion time. Show it as a stacked bar and the argument is half won.

5.2 Where a gateway genuinely adds time

Be able to list all five without notes. Conceding precisely is more persuasive than denying.

1. **One extra network hop.** Client to edge to provider. Small when the edge is near the client, larger when the client already sits in the same region as the provider.
2. **Key and credit lookups.** Cached at the edge. OpenRouter's own documentation names two conditions that make this worse: cold edge caches for the first minute or two in a new region, and more aggressive cache expiry when a credit balance is low or a key is near its configured limit. That second one is a strong "let me rule this out on your account today" line.
3. **Routing decision and schema translation.** Milliseconds, but not zero.
4. **Stream relay.** Should be near zero. If decode throughput drops through the gateway, that is a real finding worth escalating, not something to wave away.
5. **Failover on a first attempt error.** A retried request pays full latency again. This inflates the tail and is a real source of complaints. The honest response is that the counterfactual is a failed request, not a fast one.

5.3 Why the developer's benchmark is probably wrong

This section wins the developer. Do not accuse. Offer to run it together.

- **He probably was not on the same provider.** The same Claude model is served through multiple providers. If the routed call landed somewhere other than Anthropic first party, he compared two different fleets in two different regions and called it routing overhead. Default behavior load balances across top providers to maximize uptime rather than pinning one, so this is the most likely flaw by far.
- **Sequential blocks, not interleaved.** Running 200 direct then 200 routed measures time of day as much as anything.
- **Means instead of percentiles,** and a sample too small to say anything about p95.
- **One region, laptop, cold cache.** See the cold cache note above.
- **Prompt caching not matched.** If one arm benefits from a warm prefix cache and the other does not, the comparison is meaningless. Caching moves time to first token far more than a routing hop does.
- **No concurrency.** Production runs concurrent. Under concurrency, rate limits and queueing dominate, which is exactly where pooled provider capacity helps rather than hurts.

5.4 The test design you propose

Same region, same prompts, same max tokens, streaming on both arms, caching configured identically, provider pinned on the routed arm, requests interleaved A then B, minimum 500 per arm, run across a full business day at production concurrency. Report p50, p95 and p99 time to first token, decode tokens per second, error rate, and retry rate.

Those last two columns are the ones you want on the slide. That is where the gateway wins and where direct has no answer.

5.5 Two features you must be able to explain cold

Provider routing

You can set an explicit priority order of providers for a model, allow only certain providers, ignore others, restrict to a single provider with fallbacks disabled, or let it load balance. If you specify nothing, requests are load balanced across top providers to maximize uptime, weighted toward lower cost. Setting an order or a sort disables load balancing. You can also sort by price, throughput or latency, and set preferred latency or throughput thresholds at specific percentiles.

For Kestrel's hot path you would pin. For the cold path you would let it balance and sort for price. That one configuration change is the concrete deliverable you hand Raj in week one.

BYOK

Kestrel keeps its own Anthropic key and its own Anthropic contract, rate limits and discounts, and still routes through OpenRouter for failover, analytics and one integration surface. It removes the middleman framing entirely. There is a monthly allowance measured in list price inference before a percentage fee applies, and the allowance is larger on Enterprise. One configuration gotcha to raise proactively: if the customer key errors, requests can fall back to shared credits unless routing is restricted to their own key.

VERIFY BEFORE YOU PRESENT
Confirm current pricing, allowances and the exact configuration field names on the OpenRouter docs the day before. Pricing changed more than once during 2026 and the company was just acquired. Anything you state from memory is a risk in front of the people who built it.

6. Published figures and how to use them

Third party benchmarks of gateway overhead are all over the map. That spread is your friend, not your problem.

Source	Reported gateway overhead
Remio comparison, 2025	Roughly 50 to 150 ms per request from the routing layer
Enterprise DNA practitioner report, June 2026	Roughly 80 to 250 ms depending on model and region
Opper router benchmark, 2026	OpenRouter measured about 70 ms <i>faster</i> to first token than calling OpenAI directly
China LLM directory, July 2026	Much larger gaps on an open weight model routed to a different backend, which is really a backend difference, not a gateway difference
OpenRouter documentation	Overhead minimized through edge compute, edge caching of key data, and optimized routing. Names cold caches and low balance conditions as the two situations that add latency.

How to use this on stage: do not lean on any single number as proof. Put the spread on one slide and make the point that the only number that counts is the one measured on Kestrel's own traffic, under their own conditions, which is exactly what you are proposing to run with Raj.

7. The demo plan

Do not put a live network call in a timed pitch. Build it in this order.

1. Run the benchmark script against your own OpenRouter account. Unpinned versus pinned versus direct, logging the provider actually used and the full time to first token distribution.
2. Run it a few times at different hours and keep the raw data.
3. Build the demo as a small interactive page driven by that real data, with three buttons: unpinned, pinned, and a modeled provider failure showing recovery. Ten seconds each.
4. Keep the script and the raw CSV as the backup and the leave behind.

You get the credibility of real measurement with none of the live call risk, and the demo is literally the artifact you are proposing Kestrel run themselves.

DO NOT OVERCLAIM

You cannot manufacture a real provider outage on demand. For the failover story, use the historical incident in the account narrative plus documented routing behavior, and say plainly that it is modeled rather than measured. Being explicit about the boundary of your data is worth more than a fake chart, and the panel will test exactly this.

8. Benchmark script: setup and run instructions

File: `or_latency_bench.py`. It runs three arms round robin: OpenRouter default routing, OpenRouter pinned to Anthropic with fallbacks off, and Anthropic direct. It records the provider that actually served every request, separates time to first byte from time to first token, and reports p50 through p99 rather than an average.

8.1 OpenRouter account setup

Step 1. Create the key. Sign in at openrouter.ai, go to Keys, create a key, name it something like `bench-kestrel`. Set a credit limit on the key, 10 dollars is plenty. Copy it once, you cannot see it again.

Step 2. Load credits. Settings, then Credits. Load 20 dollars in a single top up, not several small ones. There is a minimum fee per purchase, so small repeated top ups waste money.

This matters for the benchmark specifically: when a balance is low or a key is near its configured limit, edge caches expire more aggressively, which adds latency. Benchmark on a nearly empty balance and you will measure your own low balance, publish a bad number, and get taken apart in Q and A. It is also a real diagnostic line for the pitch: before we blame the hop, let me confirm your account is not in the low balance path.

Step 3. Check account wide routing settings. Settings, then Privacy. Two things there will silently change your results. Allowed providers and ignored providers apply account wide and act as a ceiling on anything set per request. Data policy filtering and zero data retention enforcement also restrict which endpoints are eligible. Leave them at defaults for the benchmark, but know they exist, because account wide ZDR and provider restriction is a real answer to a CTO compliance question.

Step 4. Get the exact provider slug. Open the model page for the Claude model you are testing. There is a copy button next to each provider name. Copy it exactly and pass it with `--pin`. Base slugs match every region and variant for that provider. A suffixed slug pins one specific region or variant.

Step 5. Verify both model strings. The OpenRouter slug and the Anthropic direct model id are different strings. Check both before you run, or the direct arm returns a 404 and you lose the run.

Step 6. Anthropic key. Create one at console.anthropic.com and load a small amount of credit. Without this there is no direct arm and the whole comparison collapses.

8.2 Running it

```
pip install requests
export OPENROUTER_API_KEY=sk-or-...
export ANTHROPIC_API_KEY=sk-ant-...

# smoke test, about 12 calls
python or_latency_bench.py --n 2 --warmup 1 --lookup-stats --yes

# the real run
python or_latency_bench.py --n 60 --warmup 3 --concurrency 4 \
  --lookup-stats --out kestrel_am
```

`--lookup-stats` is the one flag to always use. It pulls the generation record for each OpenRouter call and returns the provider name, OpenRouter's own measured latency, generation time, whether the call was BYOK, and the cost. That is your receipt when the developer says your number is wrong.

8.3 What to actually run for the deck

- Three runs at different hours, morning, midday and evening, into different output prefixes.
- One run at `--concurrency 8` for a loaded picture.
- One run with `--allow-cache` to show what prompt caching does to time to first token. That delta is usually far larger than the routing delta, which is the whole point of the "you are optimizing the wrong variable" slide.

- One run with `--skip-chain` to show the cost of a routing decision when the first choice provider cannot serve the model. Label it honestly on the slide as a provider skip, not a mid stream failover. If you call it failover and the developer catches it, you lose the room.

8.4 Flags

Flag	Effect
<code>--n</code>	Requests per arm, default 60
<code>--concurrency</code>	Parallel in flight requests, default 1
<code>--warmup</code>	Discarded requests per arm, covers cold edge caches
<code>--pin</code>	Provider slug for the pinned arm
<code>--allow-cache</code>	Stop adding a unique nonce per request, so caching can apply
<code>--skip-chain</code>	Add an arm whose first listed provider cannot serve the model
<code>--lookup-stats</code>	Fetch the generation record for every OpenRouter call
<code>--no-direct</code>	Skip the direct Anthropic arm
<code>--out</code>	Output file prefix

8.5 Output

Two files per run. `PREFIX_raw.csv` holds every request with timing, provider, generation id, cost and error text.

`PREFIX_summary.json` holds the per arm percentile summary and is what feeds the demo page. Key columns: `ttfb_ms` time to the first byte of the stream, `ttft_ms` time to the first actual text token, `total_ms` to the last token, `decode_tps` output tokens per second after the first token, and `gen_provider_name`, which is the provider that really served it.

9. Q and A preparation

Likely question	Where to go
What does the Stripe acquisition mean for us?	Stripe announced an agreement to acquire OpenRouter in August 2026, reported at around 7.5 billion dollars. A CTO 90 days from renewal will ask about roadmap, pricing and neutrality. Have a straight answer ready. Do not pretend it did not happen and do not speculate beyond what is public.
Why not just build the routing ourselves?	Scope it honestly: provider integrations and their drift, failover logic, retry and backoff, spend attribution, per model observability, and someone on call for it. Then ask what that engineering time is worth against the fee, and note they would still be maintaining it in two years.
The platform fee is real money. Justify it.	Split the workloads. Put BYOK on the hot path where they already have a commit, keep shared credits as a funded fallback, and show the fee against the cost of one unresolved incident plus the engineering time to replace what they would lose.
How do we know our data is not going somewhere we did not approve?	Account wide provider restriction, data policy filtering, per request and per model group ZDR enforcement, and enterprise in region routing. This is a genuine strength, use it.
Your failover chart is not a real outage.	Agree immediately. Say it is modeled from documented routing behavior and their own incident, and offer to instrument the real thing over the next 30 days. Conceding fast here builds more trust than any slide.
What do you want from us today?	Have the ask ready in one sentence, with the 30/60/90 and named owners behind it.

10. Traps to avoid

- Do not bash going direct. Direct is the right answer for part of their traffic and saying so is what makes the rest of your case

credible.

- Do not claim zero overhead.
- Do not spend eight minutes on the OpenRouter feature list.
- Do not lose the developer after slide four. He is either your champion or the person who writes the memo.
- Do not present a number you cannot source or reproduce.
- Do not finish without asking for the renewal out loud.

11. Next steps

1. Set up the OpenRouter and Anthropic keys, load credits, verify both model slugs.
2. Smoke test the script, then run the full set: three time of day runs, one loaded run, one cached run, one skip chain run.
3. Send the summary JSON files for the demo page build and the slide narrative.
4. Build the deck against the real numbers, then rehearse to 15 minutes with a timer.
5. Re verify pricing, allowances and configuration field names on the docs the day before.

Sources for the published figures in section 6: Remio OpenRouter versus Claude direct comparison; Enterprise DNA practitioner report, June 2026; Oppen LLM router latency benchmark, 2026; China LLM directory routing overhead snapshot, July 2026; OpenRouter documentation on latency and performance, provider routing, and BYOK. Acquisition reporting: Bloomberg, TechCrunch and CNBC, August 2026, plus OpenRouter's own announcement. Figures and pricing terms change, so verify before presenting.